

PROFILEBOOK

Capstone Project

Mentored by:

Mr. Parth Shukla

Email: ParthShukla@greatlearning.in

Name: Garaga Lakshmi Supraja

Email: suprajagaraga@gmail.com

Phone No: 9618054408

Date: 07/09/2025

Batch: WIPRO-NGA-.Net-FSDA-FY26-C1

Index

S.no	TOPIC
1.	INTRODUCTION
	1.1 Motivation
	1.2 Problem Statement
	1.3 Objective of the Project
	1.4 Theory
2.	PROFILEBOOK
	2.1 Features
	2.2 Tech Stack
	2.3 Architecture Overview
	2.4 Entity Relationship Diagram (ERD)
3.	REQUIREMENTS AND RESULTS
	3.1 System Requirements
	3.2 Case Diagrams
	3.3 Implementation
	3.4 Results
4.	CONCLUSION

ABSTRACT

This document provides a detailed overview of the ProfileBook application, an online social media platform that enables users to create posts, send messages, comment, and report inappropriate behavior. Admins manage users, approve posts, and handle reports. The project utilizes Angular, ASP.NET Core MVC, SQL Server, and SignalR for real-time communication.

CHAPTER 1: INTRODUCTION

1.1 Motivation

ProfileBook connects users virtually, enabling easy interaction through posts and messaging while maintaining a secure and moderated environment.

1.2 Problem Statement

Users face fragmented services when it comes to social interaction, requiring multiple apps for messaging, posting, and reporting. ProfileBook addresses this by offering a unified platform.

1.3 Objective of the Project

- Secure authentication for users and admins.
- Post creation and admin approval.
- Commenting and liking posts.
- Real-time messaging with SignalR
- Reporting system for inappropriate content.
- Image storage and management.

1.4 Theory

The project is designed to provide seamless user experience using Angular for frontend, ASP.NET Core MVC Web API for backend, and SQL Server database. JWT is used for secure authentication, while EF Core manages the database interactions.

CHAPTER 2: PROFILEBOOK

2.1 Features

1. User Authentication (Login, Register, Logout)
2. Post Creation and Approval Process
3. Comment and Like Posts
4. Messaging between Users (SignalR)
5. Reporting UsersSearch for Other Users
6. Admin User and Post Management
7. Group Creation and Management

2.2 Tech Stack

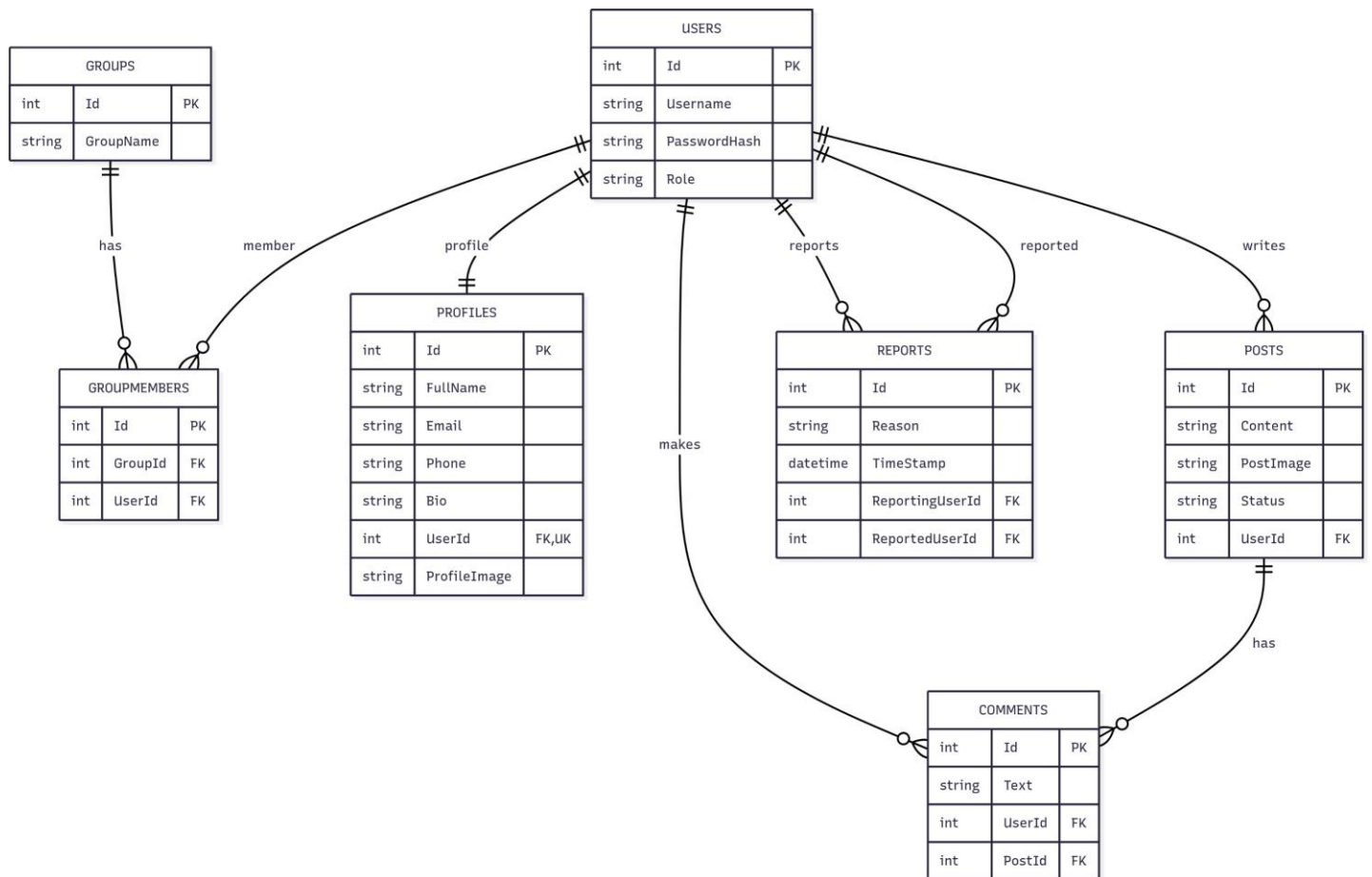
Frontend (Angular): Angular Router for navigation, Axios for API requests, Context API or Redux for state

- Backend: ASP.NET Core MVC with Web API
- Database: SQL Server (Entity Framework Core ORM)
- Image Storage: Server folder
- API Documentation: Swagger
- Real-Time Communication: SignalR (Optional)

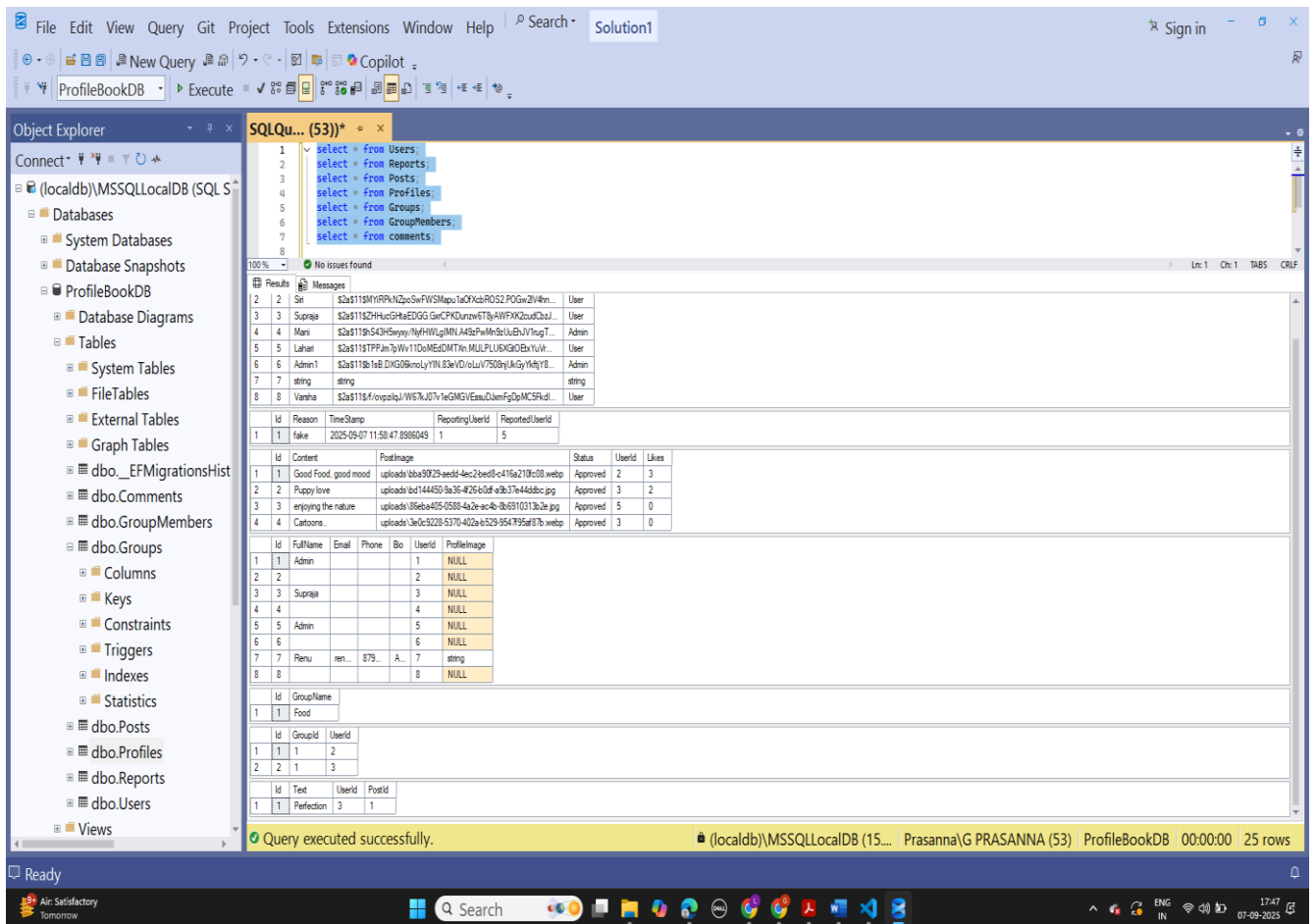
2.3 Architecture Overview

Frontend (Angular): Angular Router for navigation, Axios for API requests, Context API or Redux for state management. Backend (ASP.NET Core MVC): JWT Authentication, EF Core for DB interactions, Web API controllers (UserController, PostController, etc.). Database (SQL Server): Tables - Users, Posts, Comments, Messages, Reports, Groups, GroupMembers, Profiles. Image Upload: Handled via IFormFile, images stored in server directories. Notifications: Optional real-time notifications using SignalR.

2.4 Entity Relationship Diagram (ERD)



Tables



The screenshot shows the Microsoft SQL Server Enterprise Edition interface. The Object Explorer on the left displays the database structure for 'ProfileBookDB', including tables like 'dbo.Users', 'dbo.Reports', 'dbo.Posts', 'dbo.Profiles', 'dbo.Groups', 'dbo.GroupMembers', and 'dbo.Comments'. The Query Editor in the center contains a SQL query that joins these tables. The Results pane on the right shows the output of the query, which is a table with columns: Id, Reason, TimeStamp, ReportingUserId, ReportedUserId, Content, PostImage, Status, UserId, and Likes. The query executed successfully, returning 25 rows.

```
1 select * from Users;
2 select * from Reports;
3 select * from Posts;
4 select * from Profiles;
5 select * from Groups;
6 select * from GroupMembers;
7 select * from Comments;
```

Id	Reason	TimeStamp	ReportingUserId	ReportedUserId
1	fake	2025-09-07 11:50:47.8986049	1	5

Id	Content	PostImage	Status	UserId	Likes
1	Good Food, good mood	uploads/bba59729-aedd-4ec2-beed0-c416a210c108.webp	Approved	2	3
2	Puppy love	uploads/bd144450-9a36-426b-b0df-a3b37e44d4db.jpg	Approved	3	2
3	enjoying the nature	uploads/05eba405-0588-4a2e-ac-b5-8b9310313b2e.jpg	Approved	5	0
4	Cartoons	uploads/1a0c5228-5370-402a-b529-954795af87b.webp	Approved	3	0

Id	FullName	Email	Phone	Bio	UserId	ProfileImage
1	Admin				1	NULL
2					2	NULL
3	Supraja				3	NULL
4					4	NULL
5	Admin				5	NULL
6					6	NULL
7	Renu	ren...	879...	A...	7	string
8					8	NULL

Id	GroupName
1	Food

Id	GroupId	UserId
1	1	2
2	1	3

Id	Text	UserId	PostId
1	Perfection	3	1

Query executed successfully. (localdb)\MSSQLLocalDB (15... Prasanna\G PRASANNA (53) ProfileBookDB 00:00:00 25 rows

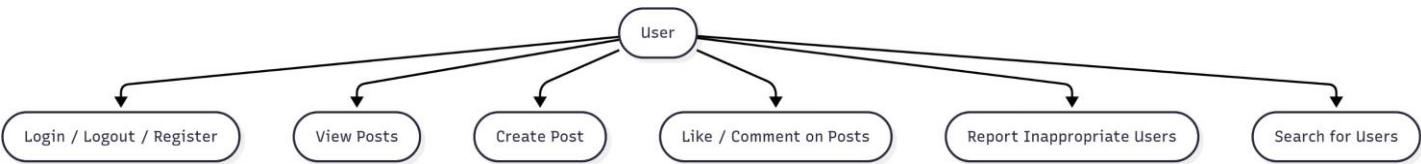
CHAPTER 3: REQUIREMENTS AND RESULTS

3.1 System Requirements

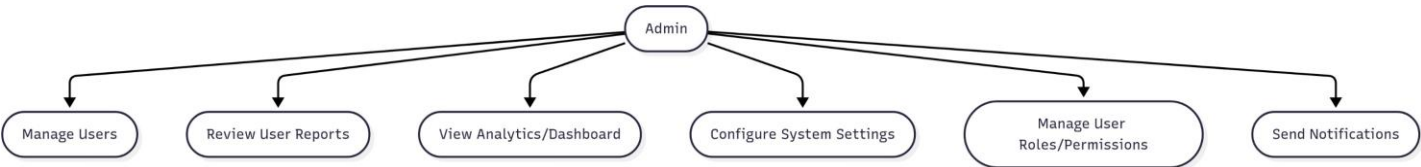
Software: Visual Studio, VS Code, SQL Server, Angular, ASP.NET Core MVC, EF Core. Hardware: Minimum 4 GB RAM, Intel/AMD dual-core, 500 MB free disk space.

3.2 Case Diagrams

User Operations Use Case Diagram



Admin Operations Use Case Diagram



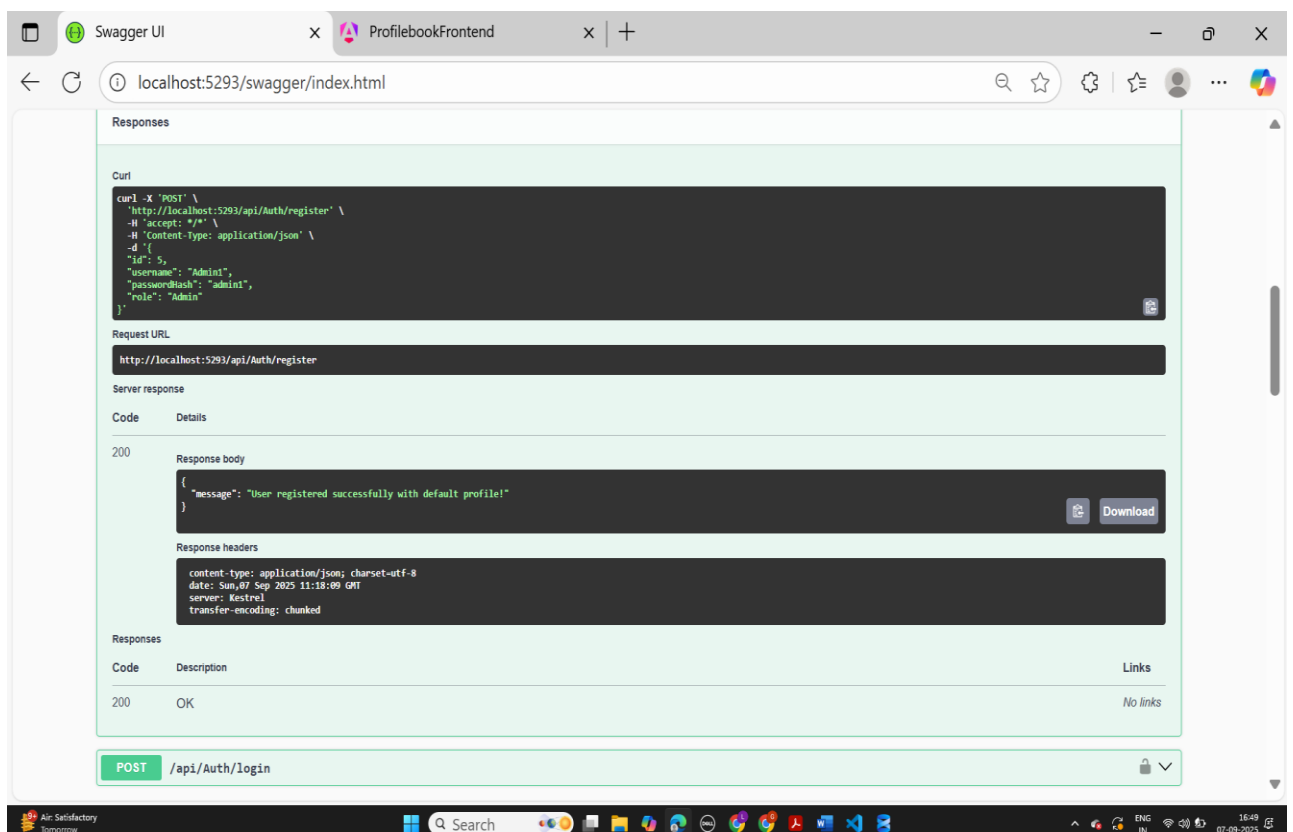
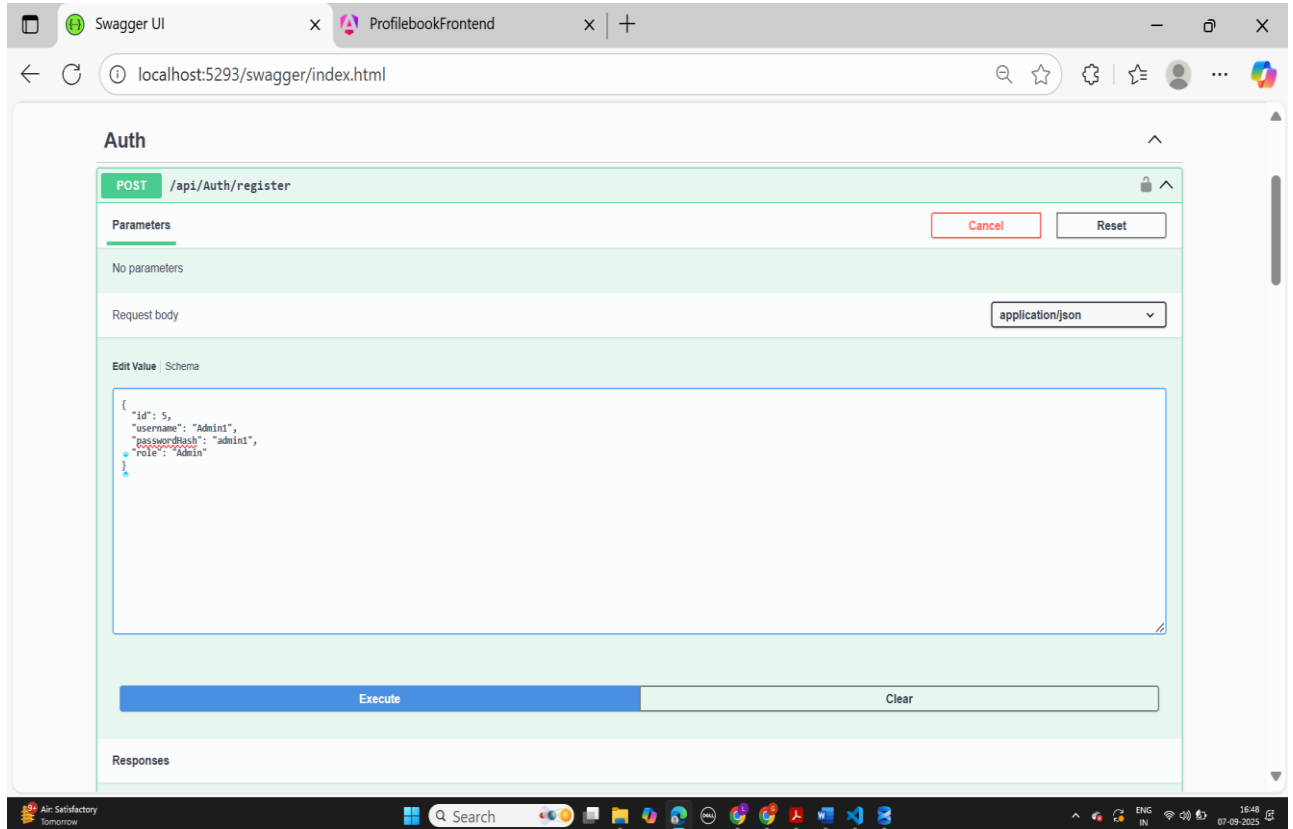
3.3 Implementation

Backend Implementation: Developed Web API endpoints, JWT Authentication, SignalR for messaging, CRUD for Posts, Comments, Users, Reports.

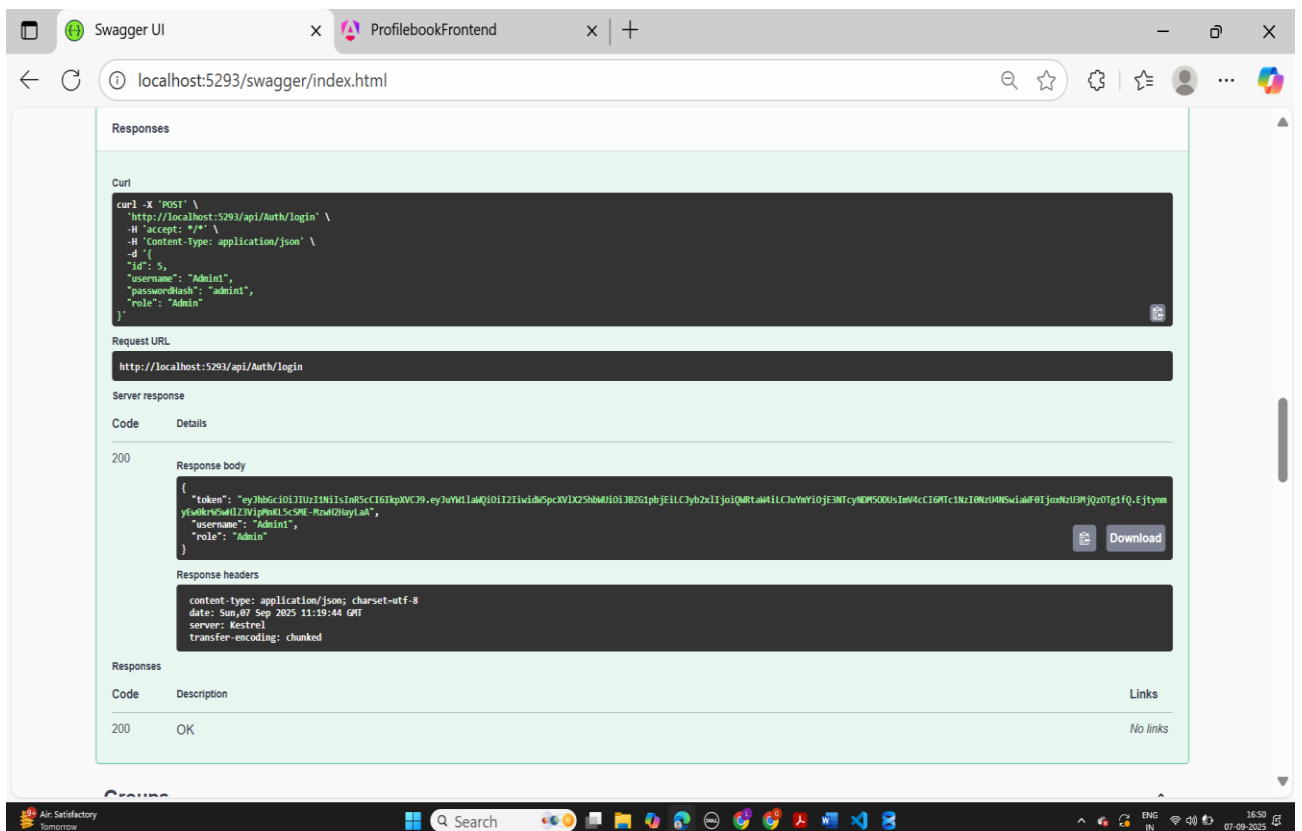
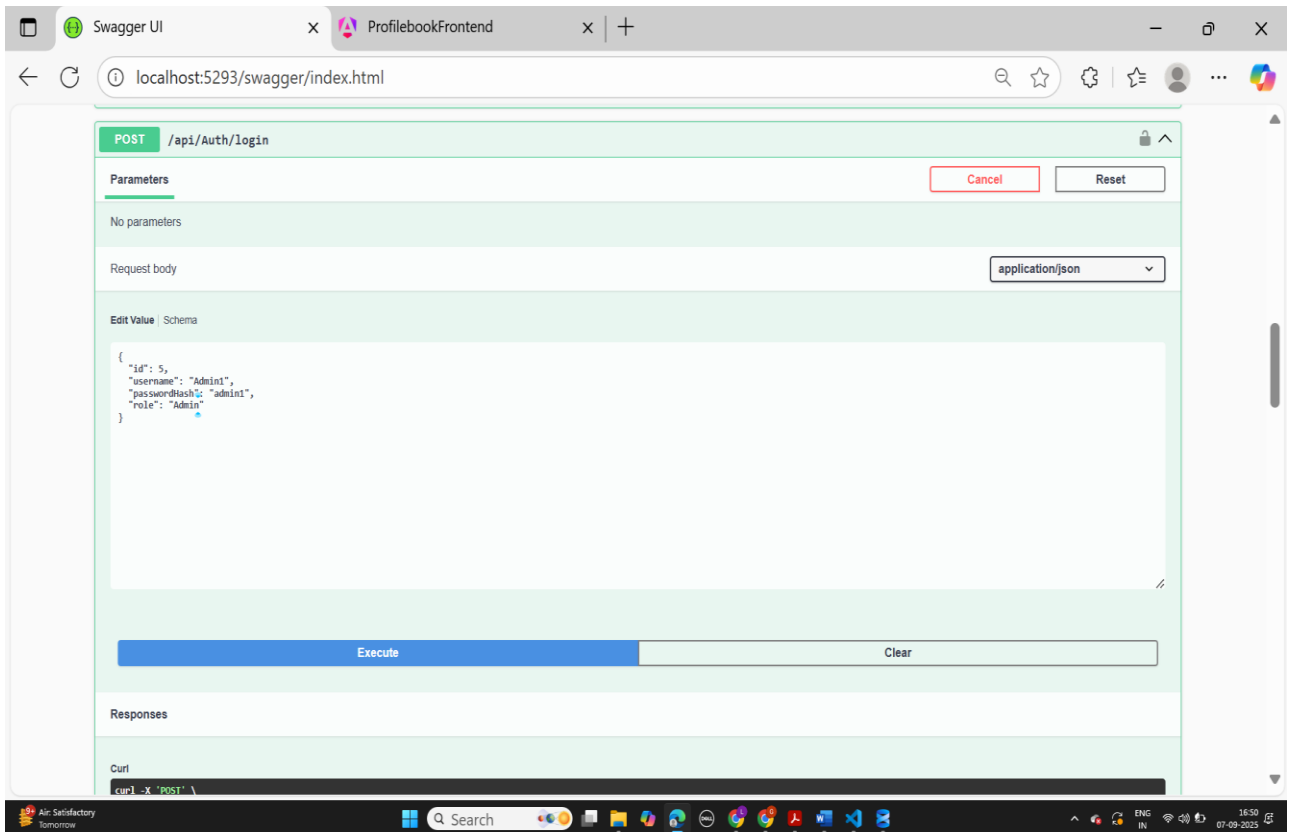
Frontend Implementation :Context API state management.

Implementation Screenshot

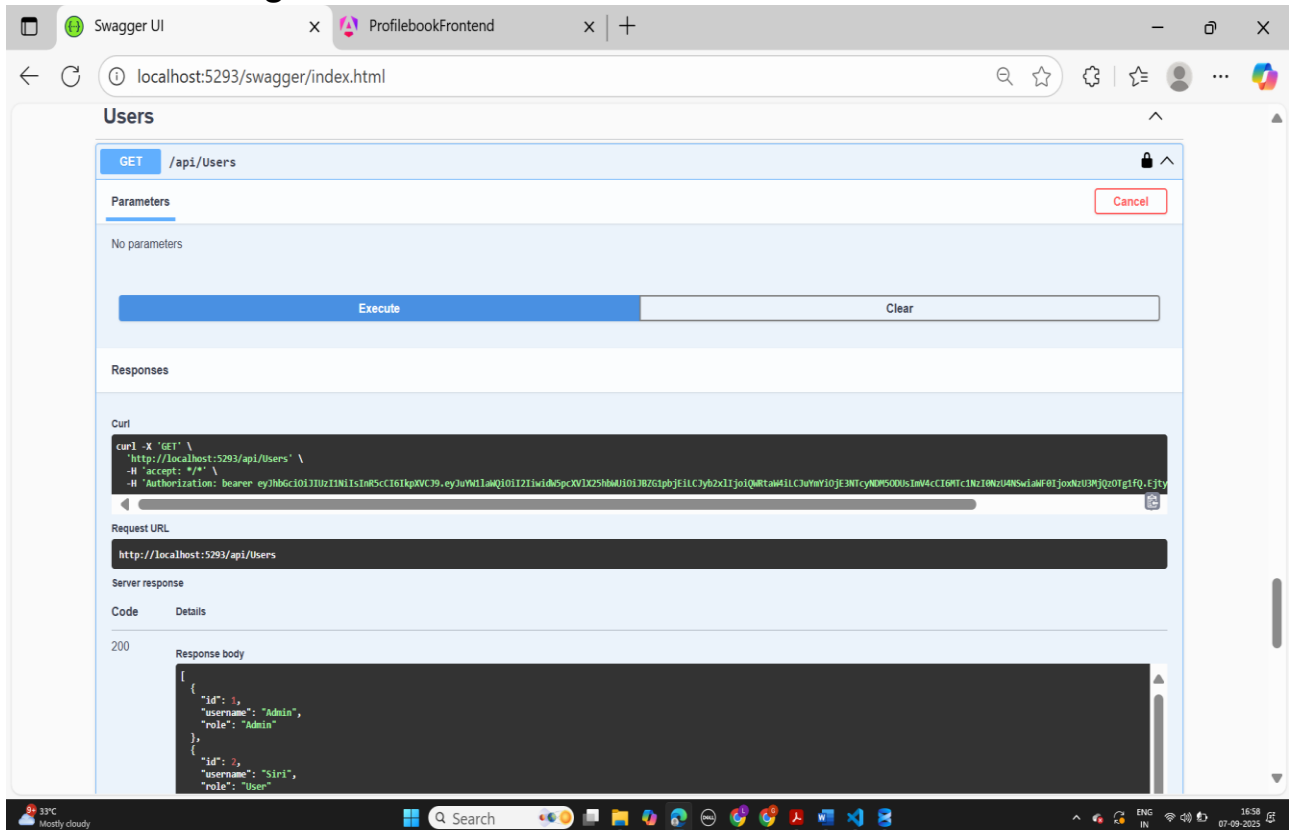
Register API



Login API



Admin fetching users



This screenshot shows the Swagger UI for the endpoint `GET /api/Users`. The interface includes a "Parameters" section with a "Cancel" button, an "Execute" button, and a "Clear" button. The "Responses" section shows a 200 status code with a response body containing a list of users.

Parameters

No parameters

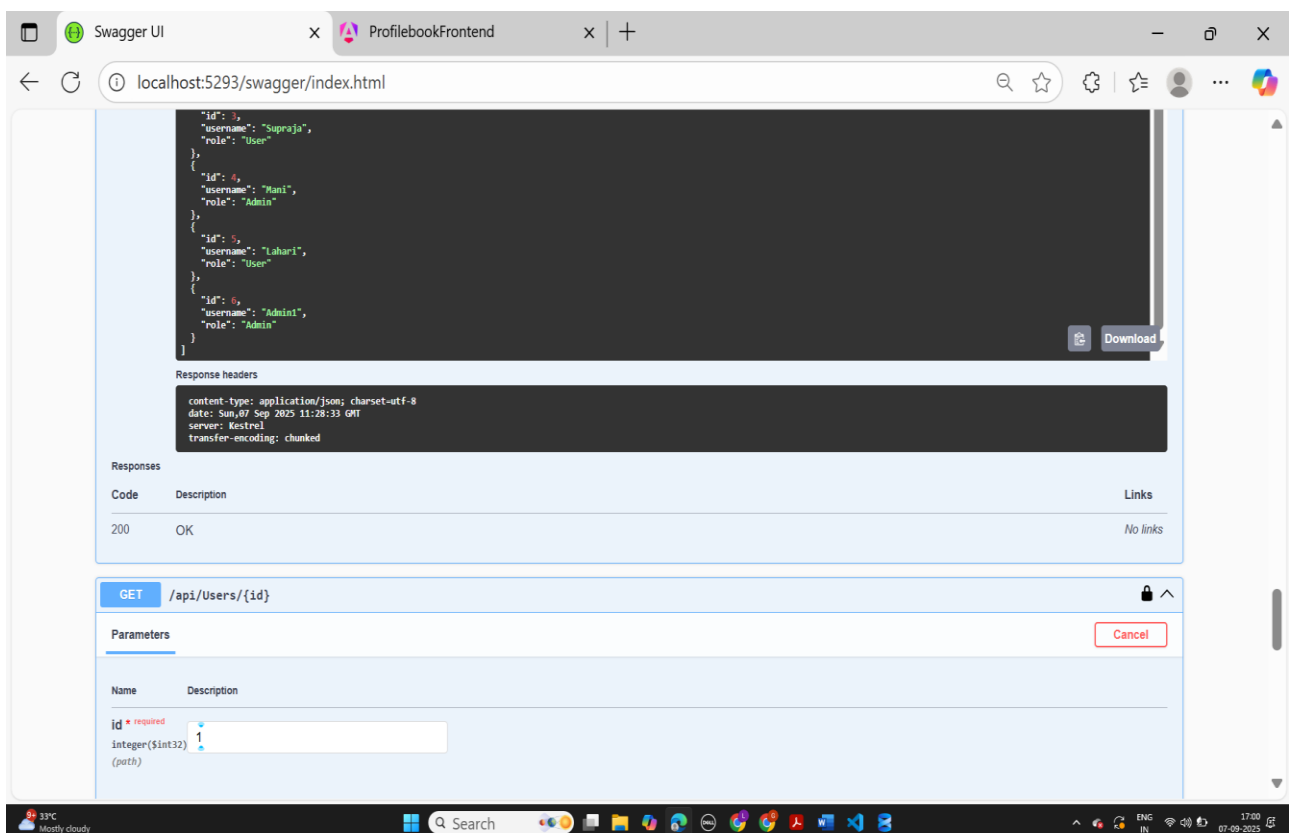
Execute **Clear**

Responses

200

Response body

```
{
  "id": 1,
  "username": "Admin",
  "role": "Admin"
},
{
  "id": 2,
  "username": "Siri",
  "role": "User"
}
```



This screenshot shows the Swagger UI for the endpoint `GET /api/Users/{id}`. The interface includes a "Parameters" section with a "Cancel" button. The "Responses" section shows a 200 status code with a response body containing a list of users. The "Response headers" section shows the content-type, date, server, and transfer-encoding.

Parameters

No parameters

Execute **Clear**

Responses

200

Response body

```
{
  "id": 1,
  "username": "Supraja",
  "role": "User"
},
{
  "id": 4,
  "username": "Mani",
  "role": "Admin"
},
{
  "id": 5,
  "username": "Lahari",
  "role": "User"
},
{
  "id": 6,
  "username": "Admin",
  "role": "Admin"
}
}
```

Response headers

```
content-type: application/json; charset=utf-8
date: Sun, 07 Sep 2025 11:28:33 GMT
server: Kestrel
transfer-encoding: chunked
```

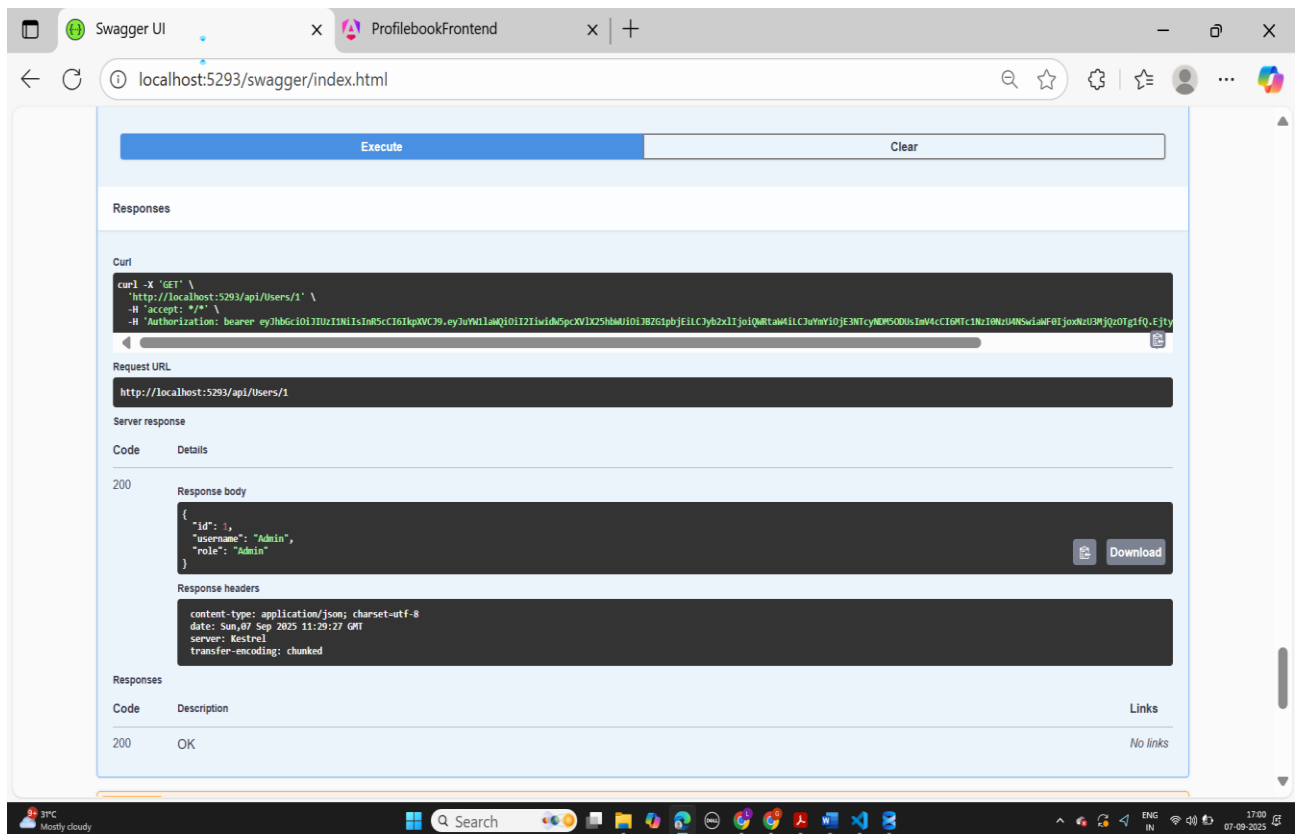
Parameters

Name **Description**

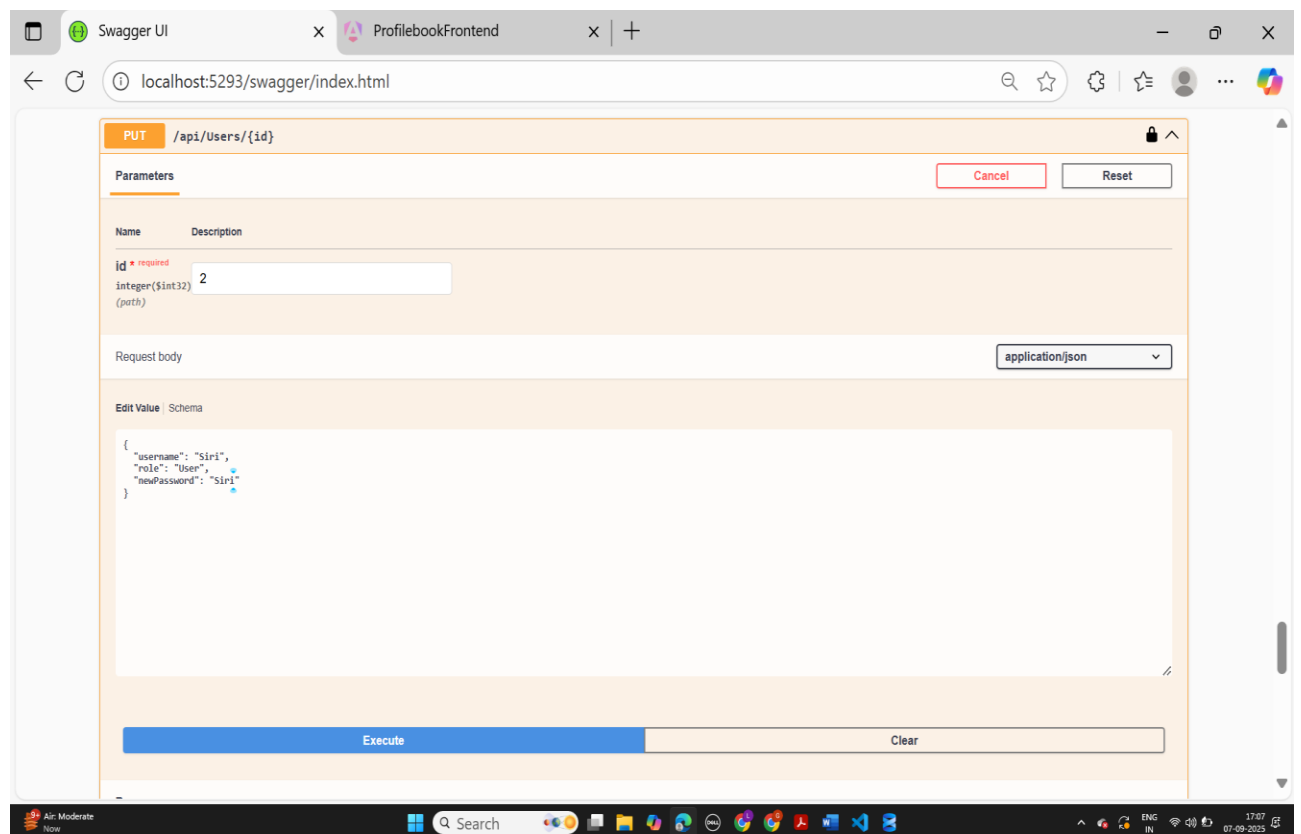
id * required
integer(\$int32)
(path)

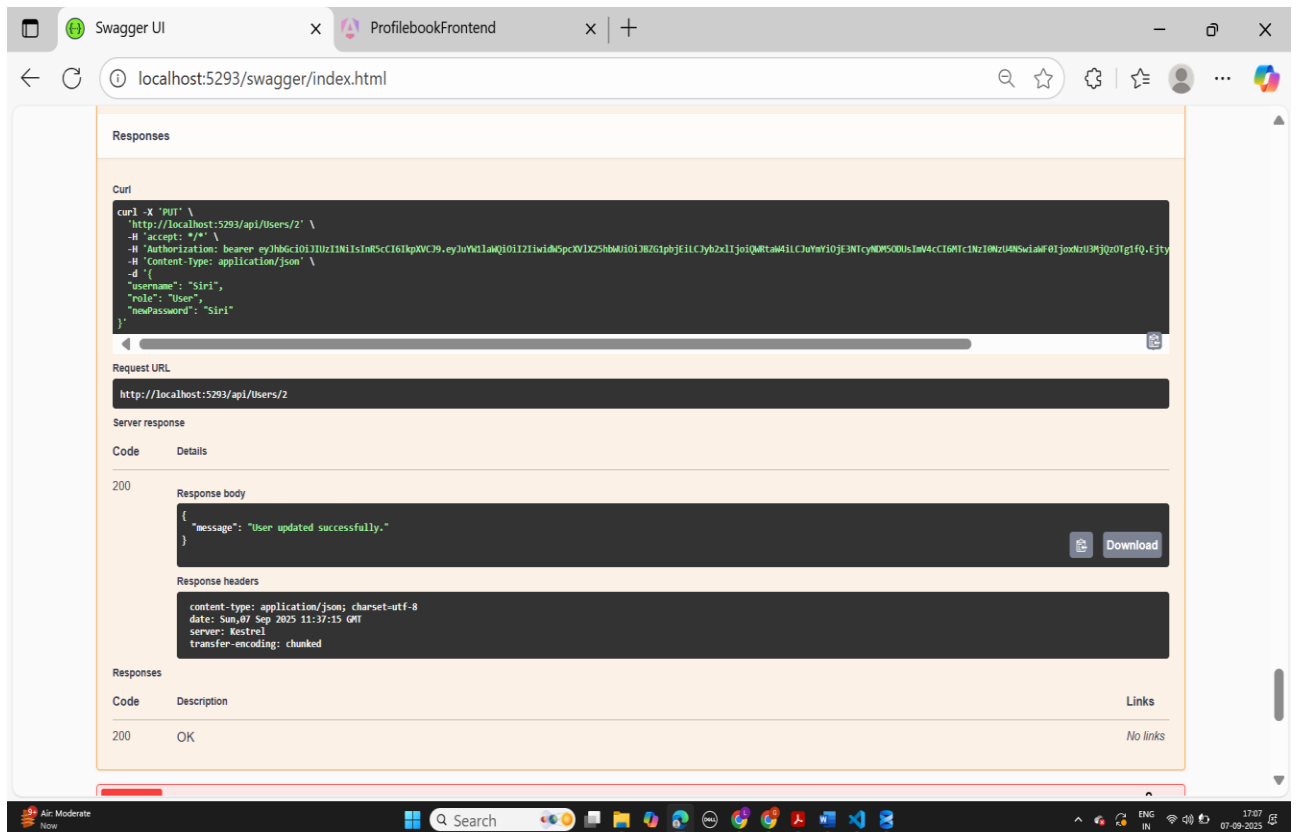
1

Fetching users by id

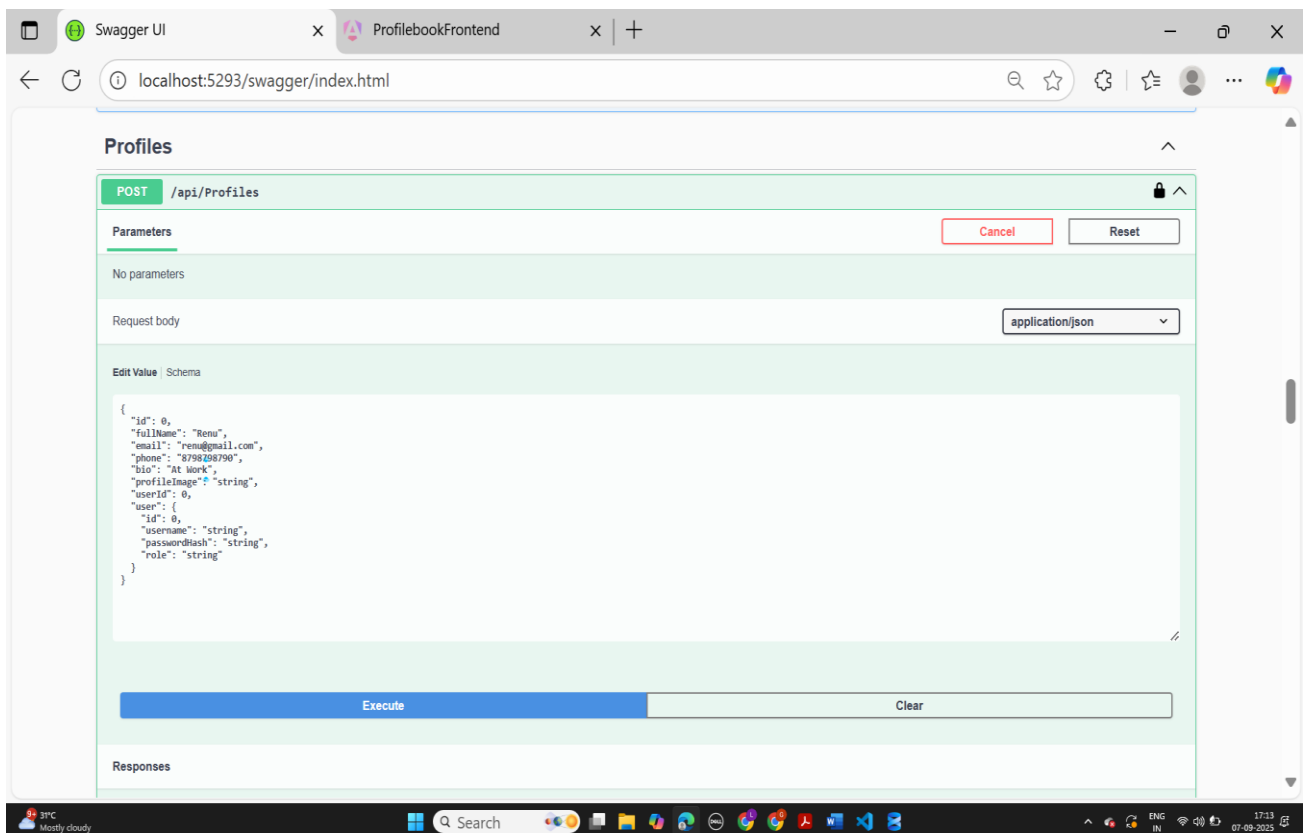


Updating user data





Creating Profile API



Swagger UI

ProfilebookFrontend

localhost:5293/swagger/index.html

```
bio: "AI Work",
"profileImage": "string",
"userId": 0,
"user": {
  "id": 0,
  "username": "string",
  "passwordHash": "string",
  "role": "string"
}
}
```

Request URL

http://localhost:5293/api/Profiles

Server response

Code

Details

200

Response body

```
{
  "id": 7,
  "fullName": "Renu",
  "email": "renu@gmail.com",
  "phone": "8798798790",
  "bio": "AI Work",
  "profileImage": "string",
  "userId": 7,
  "user": {
    "id": 7,
    "username": "string",
    "passwordHash": "string",
    "role": "string"
  }
}
```

Download

Response headers

```
content-type: application/json; charset=utf-8
date: Sun, 07 Sep 2025 11:43:29 GMT
server: Kestrel
transfer-encoding: chunked
```

Responses

Code	Description	Links
200	OK	No links

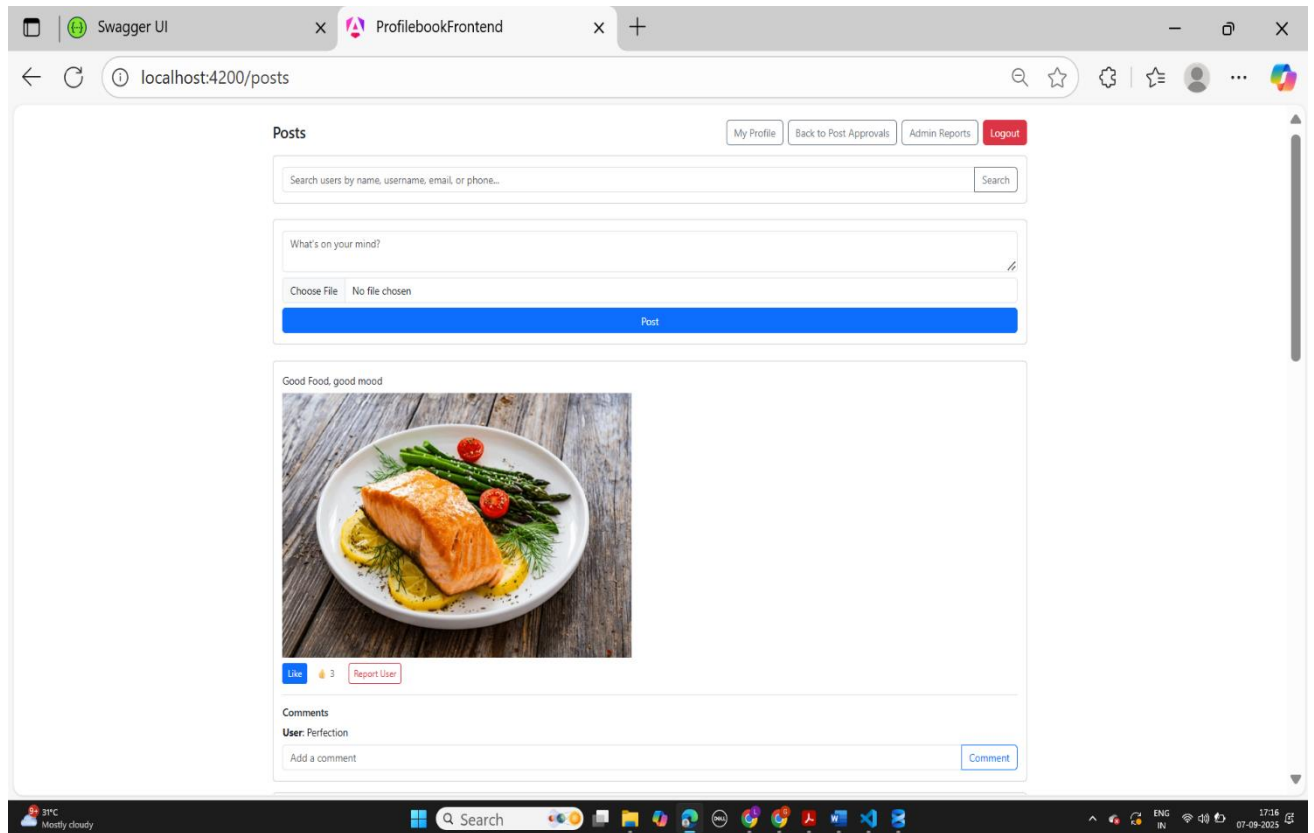
39°C Mostly cloudy

Search

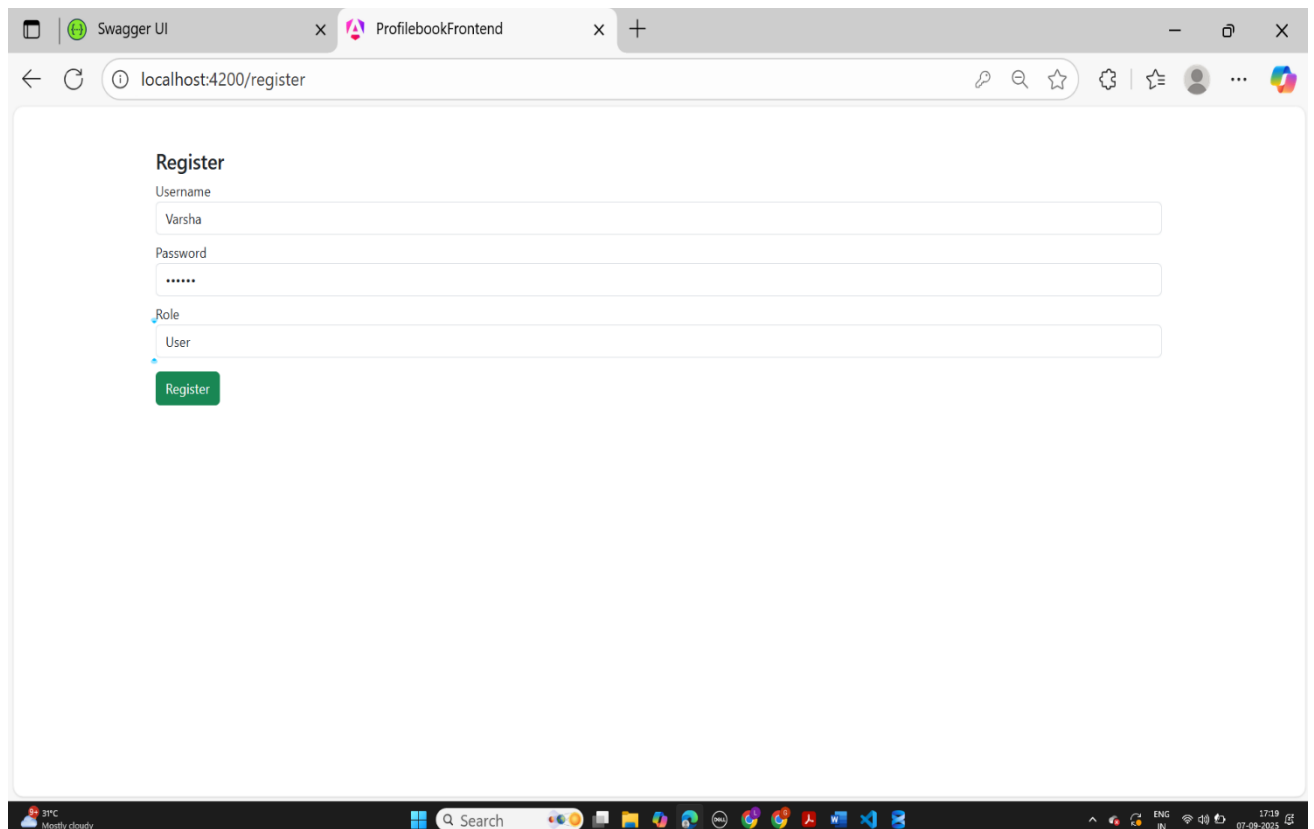
ENG IN 17:13 07-09-2025

3.4 Results

Home Page Displaying Posts:



User Registration Page:



Login Page:

Swagger UI x ProfilebookFrontend x +

localhost:4200/login

Login

Username

Password

Login

31°C Mostly cloudy Search 17:23 07-09-2023

Post Creation Interface:

Swagger UI x ProfilebookFrontend x +

localhost:4200/posts

Posts

My Profile Back to Post Approvals Admin Reports Logout

Search users by name, username, email, or phone... Search

Cartoons..

Choose File Mickey_IMG.webp

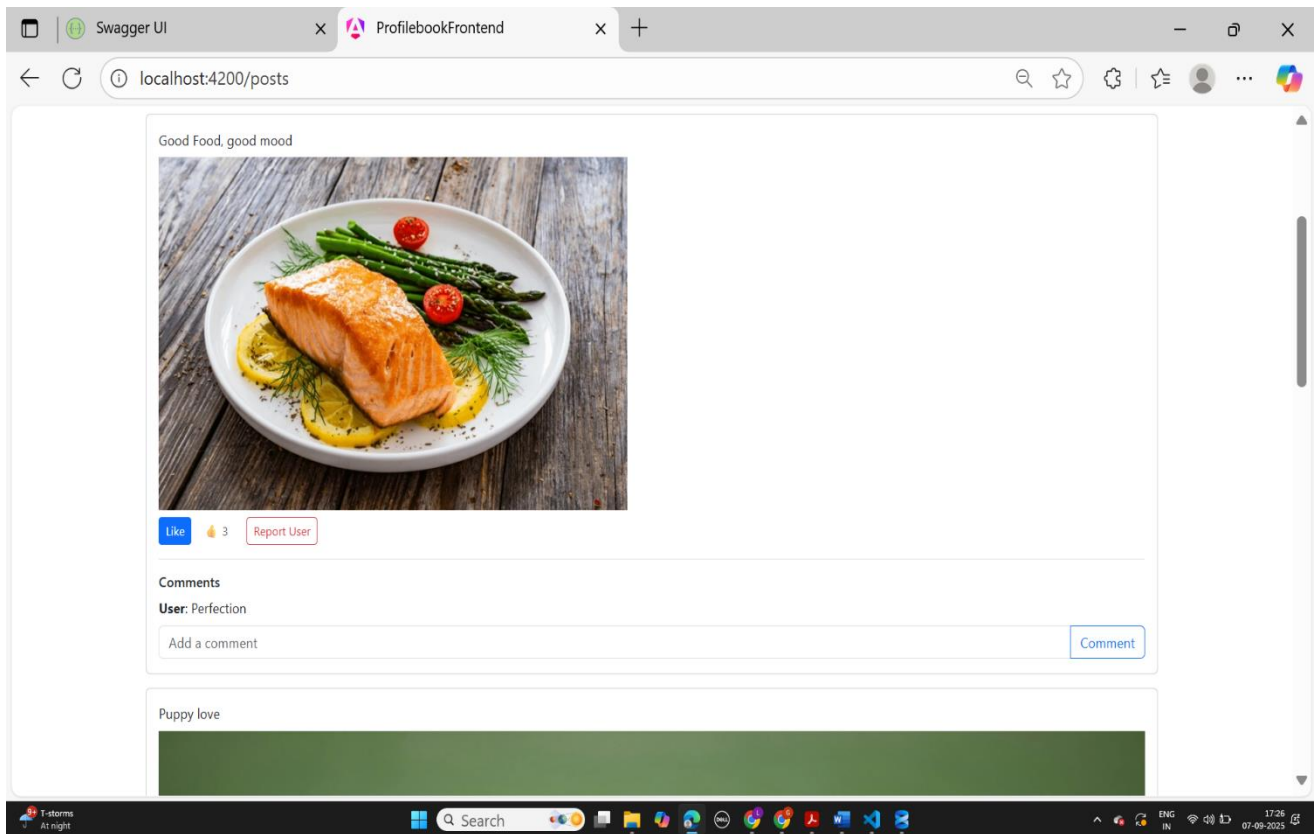
Post

Good Food, good mood

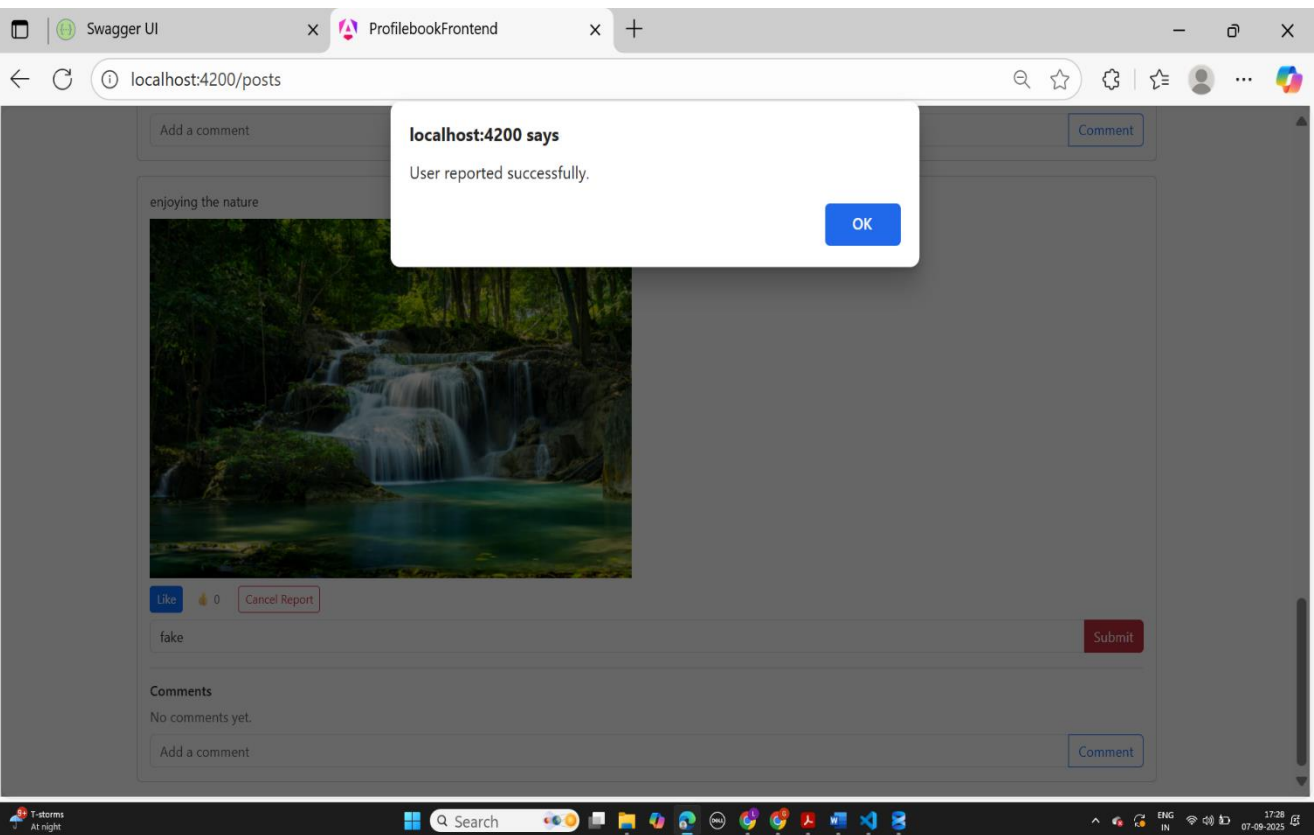


31°C Mostly cloudy Search 17:24 07-09-2023

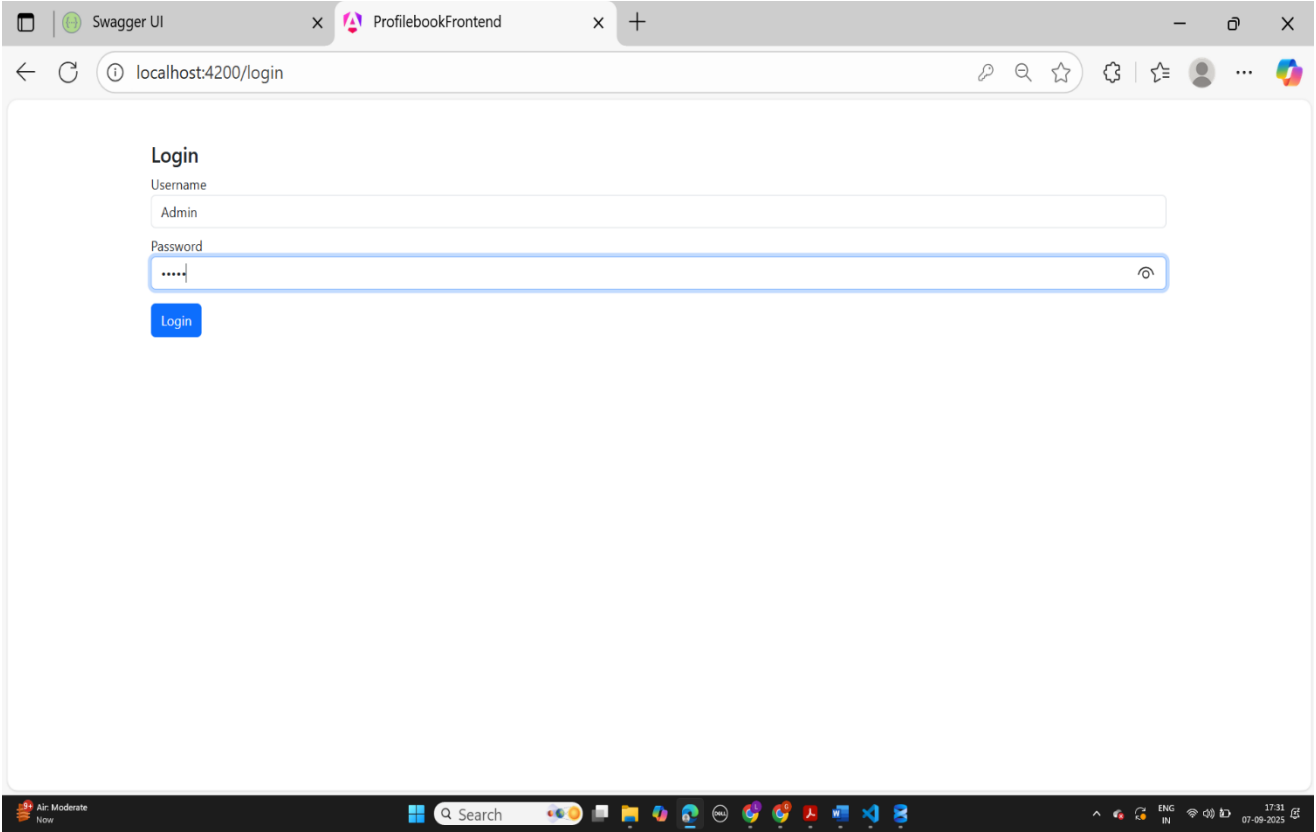
Comment/Message Interface:



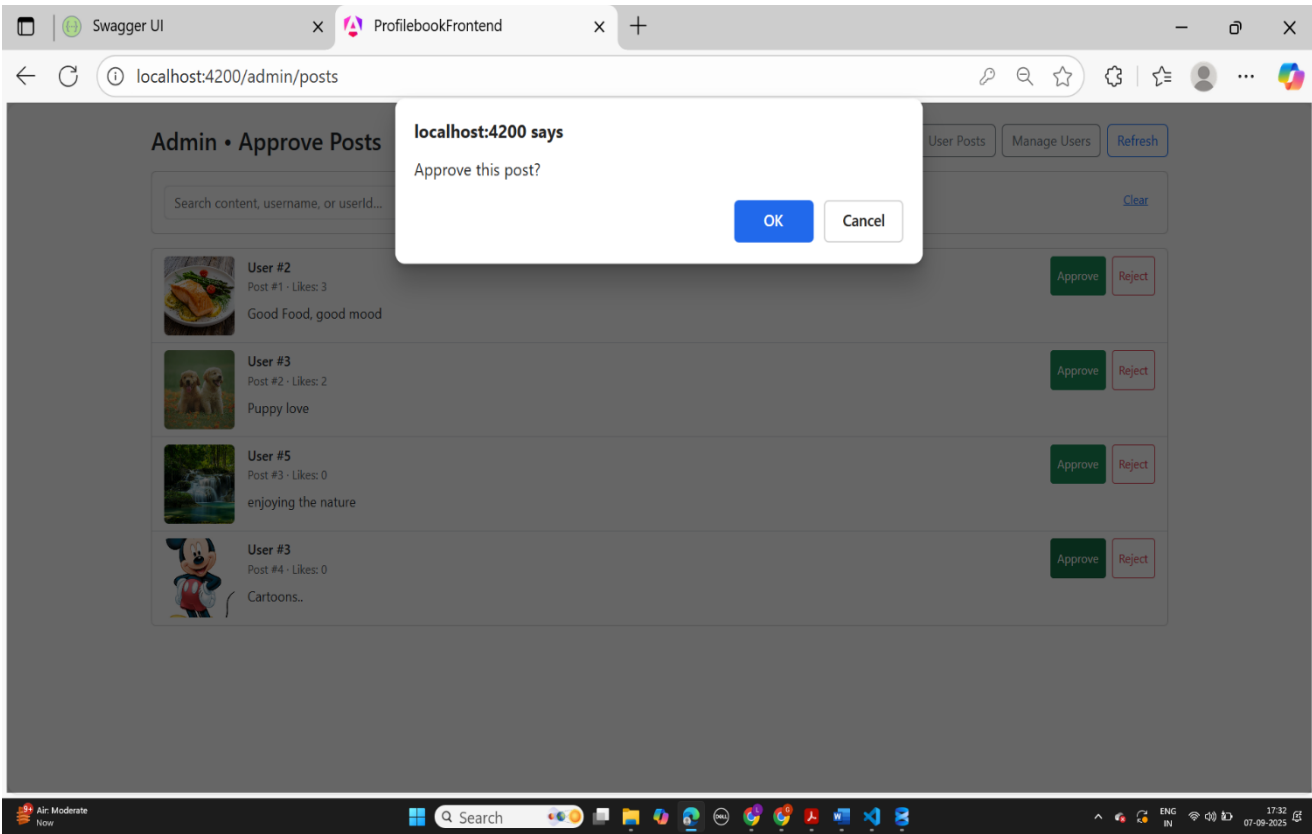
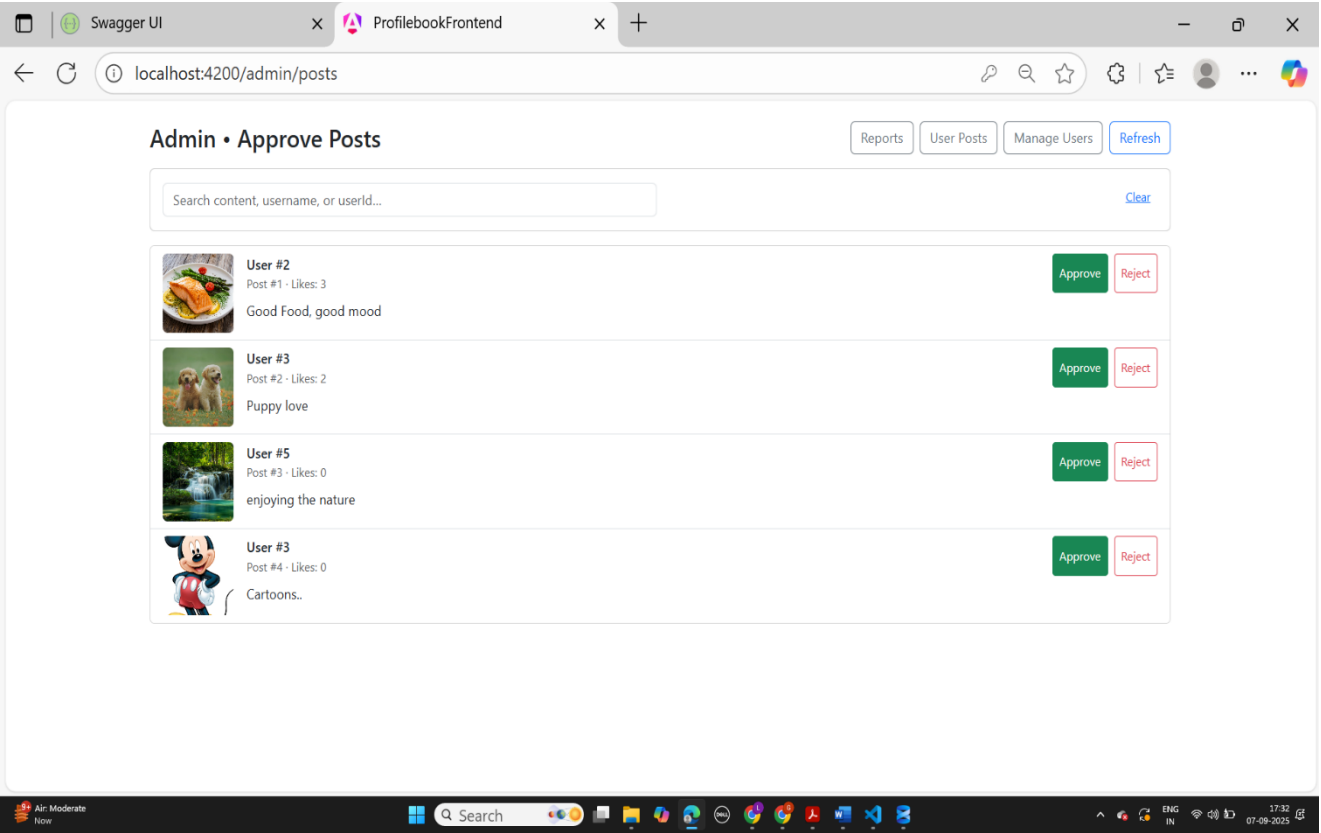
Report User Page:



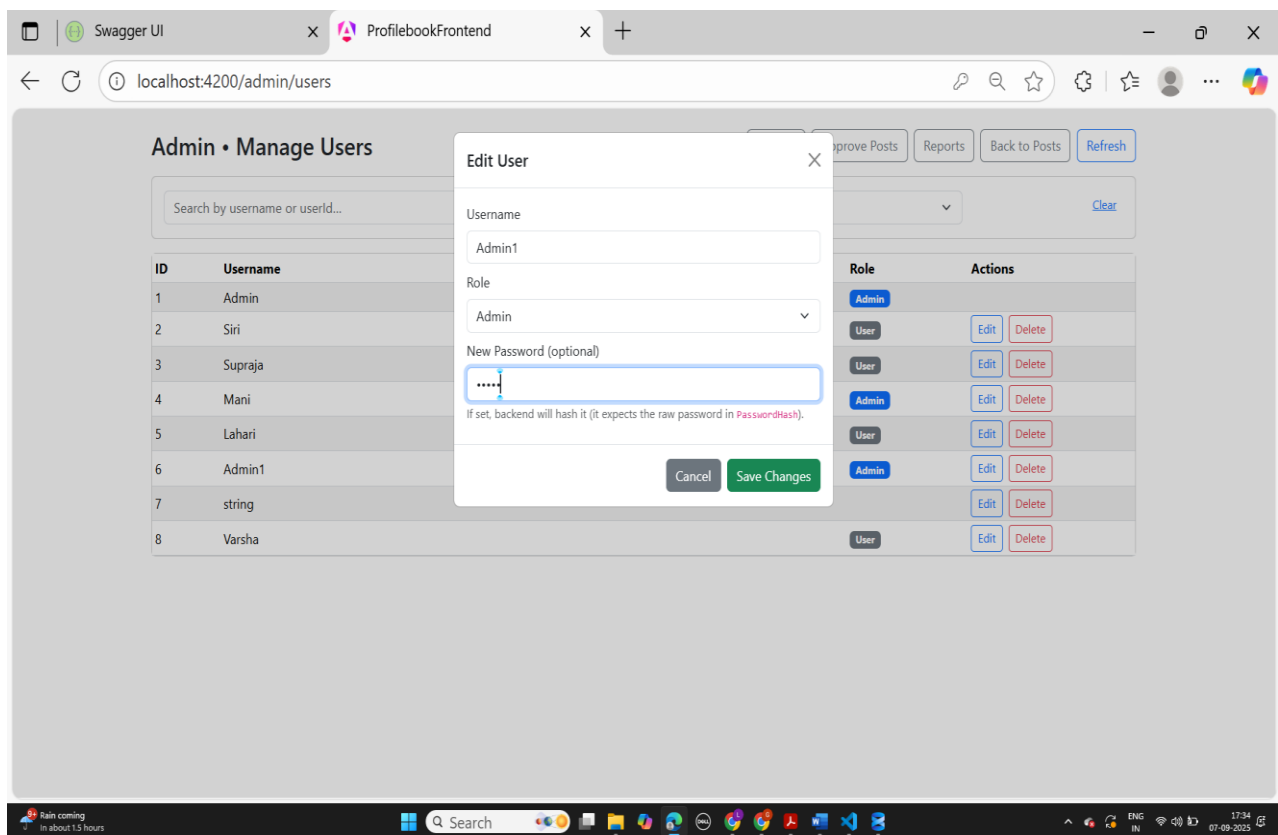
Admin Login Page:



Post Approval Dashboard:



User Management Interface:



Reported Users List:

The screenshot shows a web browser window with two tabs: 'Swagger UI' and 'ProfilebookFrontend'. The address bar displays 'localhost:4200/admin/reports'. The page title is 'Admin • Reports'. At the top right, there are three buttons: 'Back to Post Approvals', 'Back to User Posts', and a red 'Refresh' button. Below these, there are three input fields: 'Filter by reason...' (highlighted with a blue border), 'Reporting user...', and 'Reported user...'. A 'Clear filters' link is positioned below the first two fields. A table with the following data is displayed:

ID	Reason	Reporting User	Reported User	Time
1	fake	Admin	Lahari	Sep 7, 2025, 11:58:47 AM

The Windows taskbar at the bottom shows the system clock as 17:35 on 07-09-2025.

Admin managing groups

The screenshot shows the 'Admin • Groups' page. A modal dialog box is open in the center, displaying the message 'localhost:4200 says Group created.' with an 'OK' button. In the background, the 'Admin • Groups' page is visible, featuring a form with a 'Food' input field and a 'Creating...' button. Below the form, a table header is partially visible with columns 'ID', 'Group Name', 'members', and 'admins'. A message at the bottom of the table area reads 'No groups yet. Create your first group above!'. The Windows taskbar at the bottom shows the system clock as 17:38 on 07-09-2025.

Swagger UI

ProfilebookFrontend

localhost:4200/admin/groups

Admin • Groups

Approve Posts

Reports

Users

Back to Posts

Refresh

New group name (e.g., Developers, Design, QA...)

Create Group

ID	Group Name	Members	Actions
1	Food	Siri Supraja	Add Member User ID Add Tip: you can search users with Add Member button

31°C Mostly cloudy

Search

ENG IN

17:39

07-09-2025

CHAPTER 4: CONCLUSION

ProfileBook offers a comprehensive solution for online social interaction, combining ease of use, security, and functionality in a single platform. Future Scope: - AI-driven content moderation - Multi-language support - Push notifications - Enhanced analytics dashboard

References:

1. Microsoft ASP.NET Core Documentation
2. Angular Official Docs
3. SQL Server Documentation
5. Entity Framework Core Documentation